

Taller Práctico: División de Tareas y Comunicación vía Archivos

Monitorías de Sistemas Operativos

25 de marzo de 2026

Introducción

En este taller, consolidaremos el conocimiento sobre la creación de procesos mediante `fork()`. El objetivo es implementar un programa que divida el procesamiento de un archivo de datos entre múltiples hijos, utilizando archivos externos como mecanismo de persistencia y comunicación.

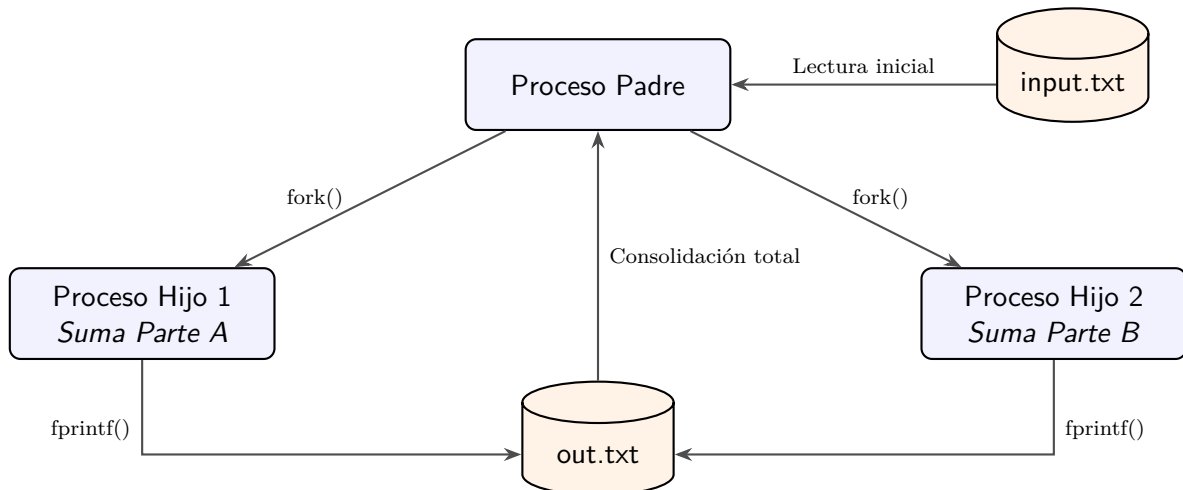
Descripción del Problema: Auditoría de Almacenes

La multinacional IBM acaba de finalizar la auditoría anual de sus infraestructuras logísticas a nivel mundial. Tras recolectar los registros de cada sede, han consolidado el valor total de existencias en un archivo maestro denominado `input.txt`. Este archivo presenta un volumen de datos crítico: la primera línea indica el número total de registros n , seguida de n valores enteros que representan el capital de cada almacén individual.

El departamento de ingeniería ha determinado que procesar esta información de forma puramente secuencial es inaceptable, ya que los tiempos de espera y el consumo monohilo de recursos retrasan la toma de decisiones corporativas. Por ello, se le ha otorgado acceso a las estaciones de trabajo de alto rendimiento con una consigna clara: paralelizar.

Como junior developer, su misión es diseñar una herramienta que aproveche la arquitectura del sistema mediante el uso de la llamada al sistema `fork()`. El objetivo es generar procesos asíncronos que compartan la carga de trabajo, sumen las secciones asignadas del inventario y retornen un resultado consolidado de forma ultraeficiente. El destino de la auditoría financiera está en sus manos; asegúrese de que la coordinación de estos procesos sea perfecta.

Esquema del Laboratorio



Responsabilidades de los Procesos

Para que el flujo sea exitoso, cada proceso debe cumplir un rol específico:

El proceso padre deberá:

1. Validar que el nombre del archivo de entrada sea pasado **obligatoriamente** como argumento por la línea de comandos (`argc` y `argv`).
2. Leer en un vector los datos del archivo de entrada indicado.
3. Estimar los índices (principio y fin) del vector sobre los cuales cada proceso hijo trabajará (usando un *delta* o *chunk*).
4. Crear los procesos hijos mediante `fork()`.
5. Esperar la terminación de los hijos mediante `wait()`.
6. Imprimir el resultado final consolidado.

Los procesos hijos deberán:

1. Recorrer el vector en las posiciones indicadas por el padre.
2. Estimar la suma de las posiciones recorridas.
3. Escribir el resultado parcial en el archivo de salida `out.txt`.

Gestión Crítica de Procesos

Para garantizar la robustez del sistema, debe prestar especial atención a:

1. **Identificación:** Use condicionales para asegurar que los hijos solo ejecuten su tarea asignada y finalicen sin continuar la ejecución del código destinado al padre.
2. **Zombies y Huérfanos:**
 - El padre debe invocar `wait()` para recolectar el estado de finalización de sus hijos. No hacerlo generará procesos en estado `defunct` (zombies).

Funciones de Apoyo

Se proporcionan las siguientes funciones para integrarlas en su desarrollo.

Utilidad de Error

```
1 void error(char *msg) {  
2     perror(msg);  
3     exit(-1);  
4 }
```

Lectura de Datos

```
1 int read_numbers(const char *filename, int **vec) {  
2     int c, number, totalNumbers;  
3     FILE *infile = fopen(filename, "r");  
4     if(!infile) error("Error opening input file");  
5  
6     fscanf(infile, "%d", &totalNumbers);  
7     *vec = (int *)calloc(totalNumbers, sizeof(int));  
8     if(!*vec) error("Error allocating memory");  
9  
10    for(c = 0; c < totalNumbers; c++){  
11        if(fscanf(infile, "%d", &number) != EOF){  
12            (*vec)[c] = number;  
13            printf("%d\n", number); // Debug purposes  
14        }  
15    }  
16    fclose(infile);  
17    return c;  
18 }
```

Cálculo de Suma Final

```
1 int read_total() {
2     FILE *infile;
3     int sum_1 = 0, sum_2 = 0, total = 0;
4     infile = fopen("out.txt", "r");
5     if (!infile) error("Error opening output file for reading");
6
7     fscanf(infile, "%d", &sum_1);
8     fscanf(infile, "%d", &sum_2);
9
10    total = sum_1 + sum_2;
11
12    fclose(infile);
13    return total;
14 }
```

Consideraciones Importantes

Comunicación y Paso por Referencia

La función `read_numbers` recibe un puntero doble (`int **vec`) para permitir que los datos leídos sean accedidos por fuera de la función mediante *paso por referencia*. Esto permite que la función reserve la memoria necesaria y el cambio persista en el `main`.

Debe invocarse haciendo uso del parámetro pasado por consola:

```
1 int *vector;
2 int total;
3
4 // Suponiendo que el archivo es el primer argumento
5 total = read_numbers(argv[1], &vector);
```

Limpieza del Entorno

Debido a que el esquema de comunicación está basado en archivos compartidos, es necesario que **antes de cada ejecución** se elimine el archivo `out.txt` para evitar leer valores de ejecuciones previas. Para esto, utilice la llamada al sistema `remove`:

```
1 remove("out.txt");
```